

ARTIFICIAL INTELLIGENCE

Comprehensive Exam Preparation Notes

Based on: Artificial Intelligence – A Modern Approach (Russell & Norvig, 3rd Ed.)

Topics Covered

1. Intelligent Agents: Agents & Environments, Rationality, PEAS, Agent Types
2. Backtracking Search: Algorithm, N-Queens Dry Run
3. Improving Backtracking: Variable & Value Ordering (MRV, Degree, LCV)
4. Further Improvements: Forward Checking & Maintaining Arc Consistency (MAC)
5. Basic Probability & Inference with Full Joint Distributions
6. Independence, Conditional Independence & Bayes Rule
7. Probabilistic Reasoning: Bayesian Networks & Markov Blankets

CHAPTER 1: Intelligent Agents

1.1 What is an Agent?

An **agent** is anything that can perceive its **environment** through **sensors** and acts upon it through **actuators**. The central idea is: $\text{percept} \rightarrow (\text{agent function}) \rightarrow \text{action}$.

□ Key Definition

An agent's behaviour is described by the agent function that maps any percept sequence to an action: $f : P^* \rightarrow A$

Agent Function vs. Agent Program

Concept	Description	Example
Agent Function	Mathematical mapping from percept sequences to actions (abstract/conceptual)	Table mapping every possible history to an action
Agent Program	Concrete implementation of the agent function running on hardware	Python/C++ code running on a robot
Percept	The agent's current input from sensors at any given instant	Camera image, temperature reading
Percept Sequence	Complete history of all percepts the agent has received	All past camera frames

1.2 Rationality – The Concept

A **rational agent** selects actions that maximise its expected **performance measure**, given the evidence from the percept sequence and any built-in prior knowledge.

Formal Definition of a Rational Agent

□ Rationality Definition

For each possible percept sequence, a rational agent selects an action that is expected to maximise its performance measure, given the evidence from the percept sequence and the agent's built-in knowledge.

Rationality vs. Omniscience vs. Perfection

Concept	Meaning	Key Point
Rationality	Maximise expected performance based on available information	Depends on percept sequence + prior knowledge
Omniscience	Knowing the actual outcome of all actions	Impossible in reality
Perfection	Always taking the best possible action	Requires omniscience; not required for rationality
Autonomy	Learning from percepts rather than relying purely on prior knowledge	A rational agent should learn

What Rationality Requires

- **Prior knowledge:** What the agent knows at design time
- **Action repertoire:** The set of actions the agent can perform
- **Percept sequence:** What has been perceived so far
- **Performance measure:** The criterion defining what 'success' means

1.3 PEAS – Specifying Task Environments

Before designing an agent, we must specify its **PEAS** description:

- **P – Performance Measure:** The criterion for evaluating success
- **E – Environment:** The external world the agent operates in
- **A – Actuators:** The mechanisms for taking actions (outputs)
- **S – Sensors:** The mechanisms for perceiving the environment (inputs)

PEAS Examples

Agent	Performance Measure	Environment	Actuators	Sensors
Taxi Driver	Safe, fast, legal, comfortable trip; profit	Roads, traffic, pedestrians, customers	Steering, accelerator, brakes, signals	Cameras, sonar, GPS, speedometer
Medical Diagnosis	Healthy patient; minimise cost/risk	Patient, hospital, staff	Display questions, tests, diagnoses	Keyboard input, medical records
Image Classifier	Correct classification rate	Grid of pixels (image data)	Classification label output	Array of pixel values
Vacuum Cleaner	Cleanliness, energy efficiency	Room with dirt and obstacles	Left, Right, Suck, Off	Dirt sensor, location sensor

1.4 The Nature of Environments

Task environments vary along several key dimensions. Understanding these dimensions is essential for choosing the right agent design.

Dimension	Types	Description	Example
Observability	Fully / Partially Observable	Can the agent access the complete state? Partial = needs memory	Chess (full) vs. Poker (partial)
Agents	Single / Multi-Agent	Is there one or more agent acting? Multi-agent may be competitive or cooperative	Crossword (single) vs. Chess (multi)
Determinism	Deterministic / Stochastic	Is the next state fully determined by current state + action?	Chess (det) vs. Taxi (stoch)

Dimension	Types	Description	Example
Episodic / Sequential	Episodic / Sequential	Episodic: current action does not affect future. Sequential: actions affect future	Image classifier (ep) vs. Chess (seq)
Dynamism	Static / Dynamic / Semidynamic	Can environment change while agent deliberates?	Crossword (static) vs. Taxi (dynamic)
Continuity	Discrete / Continuous	Are state, time, and actions discrete or continuous?	Chess (disc) vs. Taxi (cont)
Knowledge	Known / Unknown	Does the agent know the outcome of every action?	Chess (known) vs. New game (unknown)

1.5 The Structure of Agents

Agent = Architecture + Program. There are four basic agent types, in increasing order of sophistication:

1. Simple Reflex Agent

Mechanism: Selects actions based solely on the current percept, using condition-action rules (if-then rules).

Limitation: Only works in fully observable environments. Has no memory.

```
function SIMPLE-REFLEX-AGENT(percept):
    state ← INTERPRET-INPUT(percept)
    rule ← RULE-MATCH(state, rules)
    action ← rule.ACTION
    return action
```

2. Model-Based Reflex Agent

Mechanism: Maintains an internal state (model of the world) to handle partial observability.

Advantage: Can handle partially observable environments.

```
function MODEL-BASED-REFLEX-AGENT(percept):
    state ← UPDATE-STATE(state, action, percept, model)
    rule ← RULE-MATCH(state, rules)
    action ← rule.ACTION
    return action
```

3. Goal-Based Agent

Mechanism: Combines world model with goal information. Considers future consequences of actions to achieve goal.

Advantage: Flexible; can change goal without rewriting rules.

4. Utility-Based Agent

Mechanism: Uses a utility function (maps states to numbers) to choose actions that maximise expected utility.

Advantage: Handles conflicting goals and uncertain outcomes via expected utility maximisation.

5. Learning Agent

A learning agent has four components:

Component	Role
Performance Element	Selects actions (one of the four agent types above)
Critic	Evaluates how well the agent is doing and feeds back to the learning element
Learning Element	Modifies the performance element based on feedback from the critic
Problem Generator	Suggests exploratory actions to discover new knowledge

CHAPTER 2: Backtracking Search

2.1 Constraint Satisfaction Problems (CSPs) – Background

Before diving into backtracking, we need to understand the setting in which it operates: Constraint Satisfaction Problems (CSPs).

CSP Component	Formal Symbol	Description
Variables	$X = \{X_1, X_2, \dots, X_n\}$	The quantities to be determined
Domains	$D = \{D_1, D_2, \dots, D_n\}$	Possible values for each variable
Constraints	$C = \{C_1, C_2, \dots, C_m\}$	Conditions variables must satisfy
Solution	Complete & consistent assignment	All variables assigned; all constraints satisfied

2.2 Backtracking Search Algorithm

Backtracking search is a **depth-first search** that assigns values to one variable at a time and backtracks whenever it reaches a dead end (no legal values remain for some variable). The key insight is **commutativity**: since we can assign variables in any order and reach the same result, we assign **one variable at a time**, reducing the search space from $n! \cdot d^n$ to d^n leaves.

Pseudo-Code

```
function BACKTRACKING-SEARCH(csp):
    return BACKTRACK({}, csp)

function BACKTRACK(assignment, csp):
    if assignment is complete:
        return assignment
    var ← SELECT-UNASSIGNED-VARIABLE(csp)
    for each value in ORDER-DOMAIN-VALUES(var, assignment, csp):
        if value is consistent with assignment:
            add {var = value} to assignment
            inferences ← INFERENCE(csp, var, value)
            if inferences ≠ failure:
                add inferences to assignment
                result ← BACKTRACK(assignment, csp)
                if result ≠ failure:
                    return result
            remove {var = value} and inferences from assignment
    return failure
```

□ Three Key Pluggable Components

1. SELECT-UNASSIGNED-VARIABLE: Which variable to assign next? (Chapter 3) 2. ORDER-DOMAIN-VALUES: Which value to try first? (Chapter 3) 3. INFERENCE: What domain reductions can we infer? (Chapter 4)

2.3 Dry Run: N-Queens Problem

Problem: Place N queens on an N×N chessboard such that no two queens attack each other (no two queens in the same row, column, or diagonal).

Variables: $Q_1, Q_2, \dots, Q_n$ (queen in each column, value = row number)

Domains: {1, 2, 3, 4} for each queen (4-Queens example)

Constraints: $Q_i \neq Q_j$ (different rows), $|Q_i - Q_j| \neq |i - j|$ (different diagonals)

4-Queens Dry Run – Step by Step

Step	Assignment	Action	Explanation
1	$Q_1 = 1$	Assign	Try Q_1 in row 1. No conflicts yet.
2	$Q_1 = 1, Q_2 = 1$	Fail	$Q_2 = 1$ conflicts with $Q_1 = 1$ (same row)
3	$Q_1 = 1, Q_2 = 2$	Fail	$Q_2 = 2$ conflicts diagonally: $ 1-2 = 1-2 =1$
4	$Q_1 = 1, Q_2 = 3$	OK	No conflict yet. Continue.
5	$Q_1 = 1, Q_2 = 3, Q_3 = 1$	Fail	Same row as Q_1
6	$Q_1 = 1, Q_2 = 3, Q_3 = 2$	Fail	Diagonal with Q_2 : $ 3-2 = 2-3 =1$
7	$Q_1 = 1, Q_2 = 3, Q_3 = 4$	Fail	Diagonal with Q_1 : $ 1-4 = 1-3 ...$ $ 1-4 =3, 1-3 =2$. OK! But check Q_4 ...
8	$Q_1 = 1, Q_2 = 3, Q_3 = 4, Q_4 = 1$	Fail	Same row as Q_1
9	$Q_1 = 1, Q_2 = 3, Q_3 = 4, Q_4 = 2$	OK!	$Q_4 = 2$ passes all checks. SOLUTION FOUND!

□ Final 4-Queens Solution

$Q_1 = 1, Q_2 = 3, Q_3 = 4, Q_4 = 2$ Board visualisation: Col: 1 2 3 4 Row1: Q . . . Row2: . . . Q Row3: . Q . . Row4: . . Q . Verify: No two queens share a row, column, or diagonal. ✓

CHAPTER 3: Improving Backtracking Search

3.1 Variable Ordering: Minimum Remaining Values (MRV)

The **Minimum Remaining Values (MRV)** heuristic chooses the variable with the **fewest legal values** remaining in its domain. Also called the **most-constrained variable** or **fail-first** heuristic.

Why MRV Works

- Picks the variable most likely to fail soon.
- Failing early prunes larger portions of the search tree.
- If a variable has no legal values left, backtracking occurs immediately.
- Can improve performance by a factor of 1,000× or more on some problems.

MRV Example (Australia Map Colouring)

Variables: WA, NT, Q, NSW, V, SA, T. Colours: {red, green, blue}. After assigning WA=red, NT=green:

□ MRV Choice

SA's domain has been reduced to {blue} (only 1 value left). MRV selects SA next because it has fewer remaining values than Q, NSW, V. Result: SA=blue is forced. This eliminates branching entirely!

3.2 Tie-Breaking: Degree Heuristic

When MRV produces a tie (multiple variables with same domain size), the **degree heuristic** breaks ties by choosing the variable involved in the **most constraints** on other unassigned variables.

- SA in Australia has degree 5 (connected to WA, NT, Q, NSW, V).
- Choosing SA first constrains the most neighbours, most effectively pruning the tree.
- Degree heuristic is most useful at the start when all domains are full.

3.3 Value Ordering: Least-Constraining Value (LCV)

Once a variable is selected, the **Least-Constraining Value (LCV)** heuristic orders **values** to try. It prefers the value that **rules out the fewest choices** for the neighbouring variables.

LCV Intuition

□ Fail-First vs. Fail-Last

Variable ordering → FAIL-FIRST (MRV): choose variable most likely to fail to prune early. Value ordering → FAIL-LAST (LCV): choose value most likely to succeed (leaves most flexibility). Reason: We want only ONE solution, so try the most promising value first.

LCV Example

After {WA=red, NT=green}, choosing value for Q:

- **Q=blue:** Rules out the only remaining value for SA (bad choice!)
- **Q=red:** Leaves SA={blue} (one option remains). LCV prefers red.

Heuristic	Type	Principle	Effect
MRV	Variable ordering	Choose variable with fewest legal values	Fail early, prune tree aggressively

Heuristic	Type	Principle	Effect
Degree	Variable ordering (tie-break)	Choose variable with most constraints on unassigned vars	Reduce future branching factor
LCV	Value ordering	Choose value that rules out fewest neighbour values	Maximise flexibility for future assignments

CHAPTER 4: Forward Checking & Maintaining Arc Consistency (MAC)

4.1 Inference in Backtracking Search

Inference means using the current assignment to **reduce the domains** of future variables. This is the INFERENCE function in the backtracking algorithm. Two key techniques: **Forward Checking** and **Maintaining Arc Consistency (MAC)**.

4.2 Forward Checking

Whenever a variable X is assigned a value, **forward checking** looks at each unassigned variable Y that is connected to X by a constraint, and **deletes from Y 's domain** any value that is inconsistent with the value chosen for X .

Forward Checking Pseudo-Code

```
function FORWARD-CHECKING(csp, var, value):
    for each unassigned variable Y in NEIGHBORS(var):
        for each value v in DOMAIN(Y):
            if NOT consistent(var=value, Y=v, constraints):
                remove v from DOMAIN(Y)
        if DOMAIN(Y) is empty:
            return failure // Dead end detected!
    return success
```

Forward Checking Example (Australia)

After Assignment	WA	NT	Q	NSW	V	SA	T
Initial domains	R,G,B	R,G,B	R,G,B	R,G,B	R,G,B	R,G,B	R,G,B
After WA=Red	Red	G,B	R,G,B	R,G,B	R,G,B	G,B	R,G,B
After Q=Green	Red	B	Green	R,B	R,B	B	R,G,B
After V=Blue ✗	Red	B	Green	Red	Blue	[]	R,G,B

□ Key Insight

After V=Blue, SA's domain becomes EMPTY. Forward checking immediately detects this failure and backtracks, BEFORE assigning SA. This avoids wasteful search.

Limitation of Forward Checking

Forward checking detects conflicts between **one assigned and one unassigned variable**, but **not between two unassigned variables**. Arc Consistency is needed for deeper propagation.

4.3 Arc Consistency (AC-3)

An arc $(X_i \rightarrow X_j)$ is **arc consistent** if for every value of X_i there is **at least one compatible value** in the domain of X_j . The **AC-3 algorithm** enforces arc consistency across all arcs.

AC-3 Algorithm Pseudo-Code

```
function AC-3(csp):
    queue ← all arcs (Xi, Xj) in csp
    while queue is not empty:
```

```

    (Xi, Xj) ← REMOVE-FIRST(queue)
    if REVISE(csp, Xi, Xj):
        if DOMAIN(Xi) is empty:
            return false // No solution possible
        for each Xk in NEIGHBORS(Xi) - {Xj}:
            add (Xk, Xi) to queue
    return true

function REVISE(csp, Xi, Xj):
    revised ← false
    for each value v in DOMAIN(Xi):
        if no value in DOMAIN(Xj) satisfies constraint(Xi=v, Xj):
            delete v from DOMAIN(Xi)
            revised ← true
    return revised

```

AC-3 Complexity

Given n variables, each with domain size d , and c binary constraints: time complexity is $O(cd^3)$. This is because each arc can be inserted into the queue at most d times, and REVISE takes $O(d^2)$ time.

4.4 Maintaining Arc Consistency (MAC)

MAC applies **arc consistency during search** (not just as a preprocessing step). Whenever a value is assigned to variable X_i , MAC calls AC-3 starting with the arcs of the neighbours of X_i .

MAC vs. Forward Checking

Technique	Propagation Depth	Detects	Cost
Forward Checking	1 level (assigned → direct neighbours)	Conflicts between assigned & unassigned vars	Low
AC-3 (preprocessing)	All arcs globally, before search	Deeper inconsistencies before search starts	Medium
MAC (during search)	Ripple-through: propagates constraints as far as possible	Deeply nested inconsistencies during search	Higher, but saves backtracking

□ MAC Advantage

MAC is strictly more powerful than forward checking. When a variable's domain is reduced by MAC, this triggers further reductions in connected variables, cascading until no more reductions are possible. This can solve problems without any backtracking at all!

CHAPTER 5: Basic Probability & Full Joint Distributions

5.1 Basic Probability Concepts

Concept	Notation	Definition / Formula
Prior (Unconditional) Probability	$P(A)$	Degree of belief in A before any evidence. $0 \leq P(A) \leq 1$
Posterior (Conditional) Probability	$P(A B)$	Degree of belief in A given evidence B. $P(A B) = P(A \wedge B)/P(B)$
Joint Probability	$P(A \wedge B)$ or $P(A, B)$	Probability that both A and B are true simultaneously
Marginal Probability	$P(A) = \sum_b P(A, B=b)$	Obtained by summing over all values of other variables
Complement Rule	$P(\neg A) = 1 - P(A)$	Probability of A not occurring
Product Rule	$P(A \wedge B) = P(A B) \times P(B)$	Relates joint to conditional probability

5.2 Full Joint Probability Distribution

The **full joint distribution** specifies the probability of every possible combination of values for all variables. It can **answer any probabilistic query** about the domain.

Example: Toothache, Cavity, Catch

Full Joint Distribution Table

Variables: Toothache (T/F), Cavity (T/F), Catch (T/F) $\rightarrow 2^3 = 8$ entries
 $P(\text{cavity} \wedge \text{toothache} \wedge \text{catch}) = 0.108$
 $P(\text{cavity} \wedge \text{toothache} \wedge \neg \text{catch}) = 0.012$
 $P(\text{cavity} \wedge \neg \text{toothache} \wedge \text{catch}) = 0.072$
 $P(\text{cavity} \wedge \neg \text{toothache} \wedge \neg \text{catch}) = 0.008$
 $P(\neg \text{cavity} \wedge \text{toothache} \wedge \text{catch}) = 0.016$
 $P(\neg \text{cavity} \wedge \text{toothache} \wedge \neg \text{catch}) = 0.064$
 $P(\neg \text{cavity} \wedge \neg \text{toothache} \wedge \text{catch}) = 0.144$
 $P(\neg \text{cavity} \wedge \neg \text{toothache} \wedge \neg \text{catch}) = 0.576$
 Sum = 1.000 ✓

Answering Queries from Full Joint Distribution

Marginalisation: Sum over values of hidden variables to get marginal probability.

```
P(cavity) = P(cavity ∧ toothache ∧ catch)
           + P(cavity ∧ toothache ∧ ¬catch)
           + P(cavity ∧ ¬toothache ∧ catch)
           + P(cavity ∧ ¬toothache ∧ ¬catch)
           = 0.108 + 0.012 + 0.072 + 0.008 = 0.200
```

Conditional Query: Use $P(A|B) = P(A \wedge B)/P(B)$

```
P(cavity | toothache) = P(cavity ∧ toothache) / P(toothache)

P(cavity ∧ toothache) = 0.108 + 0.012 = 0.120
P(toothache) = 0.108 + 0.012 + 0.016 + 0.064 = 0.200

P(cavity | toothache) = 0.120 / 0.200 = 0.600
```

Limitation of Full Joint Distribution

For n binary variables, the full joint distribution requires **2^n entries**. For 30 variables, this is over 1 billion entries — **computationally infeasible**. We need independence to simplify.

5.3 Independence

Two variables X and Y are **independent** if knowing the value of one does not change the probability of the other.

□ Independence Conditions (Equivalent Formulations)

$P(X | Y) = P(X)$ $P(Y | X) = P(Y)$ $P(X, Y) = P(X) \times P(Y)$ All three are equivalent definitions of independence.

Example: Weather and Cavity are independent: $P(\text{cavity, weather}) = P(\text{cavity}) \times P(\text{weather})$

Effect: If n variables split into two independent groups of sizes k and $n-k$, the full joint requires $2^k + 2^{n-k}$ entries instead of 2^n — a massive saving!

CHAPTER 6: Conditional Independence & Bayes' Rule

6.1 Conditional Independence

Variables X and Y are **conditionally independent given Z** if, once we know Z , knowing X does not change our belief about Y (and vice versa).

Formal Definitions of Conditional Independence

$P(X | Y, Z) = P(X | Z)$ $\leftarrow$ X is c.i. of Y given Z
 $P(Y | X, Z) = P(Y | Z)$ $\leftarrow$ Y is c.i. of X given Z
 $P(X, Y | Z) = P(X | Z) \times P(Y | Z)$ $\leftarrow$ Joint form Notation: $X \perp Y | Z$

Dental Example of Conditional Independence

Toothache and Catch are conditionally independent given Cavity:

```
P(Toothache | Catch, Cavity) = P(Toothache | Cavity)
```

```
Intuition: If we already KNOW whether there's a cavity,
then knowing about the catch probe provides no
additional information about the toothache.
```

Naïve Bayes: Assumes all evidence variables $E_1, \dots, E_n$ are conditionally independent given the hypothesis H :

```
P(H | E_1, E_2, ..., E_n) ∝ P(H) × P(E_1 | H) × P(E_2 | H) × ... × P(E_n | H)
```

Result: Only $O(n)$ parameters needed instead of $O(2^n)$. Enormously scalable!

6.2 Conditional Dependency

Conditional independence does NOT always hold. Understanding when it breaks down is important:

- **Explaining Away:** If two causes share a common effect, they may become dependent given the effect. E.g., Burglary and Earthquake are independent but become dependent if we know Alarm is ringing.
- **Common Cause:** Two effects of a common cause are conditionally independent GIVEN the cause (e.g., Toothache and Catch are c.i. given Cavity).
- **Common Effect:** Two causes of a common effect are NOT necessarily independent given the effect (see Alarm network).

6.3 Bayes' Rule

Bayes' Rule provides a way to **update beliefs** when we receive new evidence. It is the foundation of probabilistic inference.

Bayes' Rule

$P(H | E) = P(E | H) \times P(H) / P(E)$ Where:

- $P(H | E)$ = Posterior probability of H given evidence E
- $P(E | H)$ = Likelihood: probability of evidence given hypothesis
- $P(H)$ = Prior probability of hypothesis
- $P(E)$ = Marginal likelihood (normalisation constant)

 Simplified: $P(H|E) \propto P(E|H) \times P(H)$

Worked Example: Meningitis Diagnosis

Given: $P(\text{stiff neck} | \text{meningitis}) = 0.70$, $P(\text{meningitis}) = 1/50000 = 0.00002$, $P(\text{stiff neck}) = 0.01$

```
P(meningitis | stiff neck) =  $\frac{P(\text{stiff neck} | \text{meningitis}) \times P(\text{meningitis})}{P(\text{stiff neck})}$ 
```

$$\begin{aligned} &= (0.70 \times 0.00002) / 0.01 \\ &= 0.000014 / 0.01 \\ &= 0.0014 \end{aligned}$$

Only a 0.14% chance of meningitis despite 70% sensitivity!
This is because the prior $P(\text{meningitis})$ is very small.

Applying Bayes' Rule with Multiple Evidence

When evidence variables E_1 , E_2 are conditionally independent given H :

$$P(H \mid E_1, E_2) = \alpha \times P(E_1 \mid H) \times P(E_2 \mid H) \times P(H)$$

where α is the normalisation constant: $\alpha = 1 / P(E_1, E_2)$

CHAPTER 7: Probabilistic Reasoning – Bayesian Networks

7.1 What is a Bayesian Network?

A **Bayesian network** (also called belief network or probability network) is a **directed acyclic graph (DAG)** where:

1. Each node represents a random variable (discrete or continuous).
2. Directed edges (arrows) from X to Y mean X is a direct cause/influence of Y . X is called a parent of Y .
3. Each node X_i has a Conditional Probability Table (CPT): $P(X_i \mid \text{Parents}(X_i))$.
4. The network encodes conditional independence relationships: each variable is conditionally independent of its non-descendants given its parents.

The Burglar Alarm Bayesian Network (Classic Example)

Alarm Network – Variables & Relationships

Burglary (B) $\rightarrow$ Alarm (A) $\leftarrow$ Earthquake (E) Alarm (A) $\rightarrow$ JohnCalls (J) Alarm (A) $\rightarrow$ MaryCalls (M)
 CPTs: • $P(B) = 0.001$ (prior: Burglary) • $P(E) = 0.002$ (prior: Earthquake) • $P(A|B,E)=0.95$,
 $P(A|B,\neg E)=0.94$, $P(A|\neg B,E)=0.29$, $P(A|\neg B,\neg E)=0.001$ • $P(J|A)=0.90$, $P(J|\neg A)=0.05$ • $P(M|A)=0.70$,
 $P(M|\neg A)=0.01$

7.2 The Semantics of Bayesian Networks

Full Joint Distribution from a Bayesian Network

The full joint distribution is the product of all CPT entries:

```
P(x1, x2, ..., xn) = ∏i P(xi | parents(xi))

Example (Alarm Network):
P(j, m, a, ¬b, ¬e)
= P(j|a) × P(m|a) × P(a|¬b, ¬e) × P(¬b) × P(¬e)
= 0.90 × 0.70 × 0.001 × 0.999 × 0.998
≈ 0.000628
```

Chain Rule and Conditional Independence

The chain rule for joint distributions:

```
P(x1, ..., xn) = ∏i P(xi | x1, ..., xi-1)

Bayesian network assumption (Markov condition):
P(xi | x1, ..., xi-1) = P(xi | Parents(xi))

This means each variable is conditionally independent
of all its predecessors given its parents.
```

7.3 Constructing a Bayesian Network

Steps to build a Bayesian network:

5. **Identify variables:** List all relevant random variables.

6. **Determine structure:** Add an arrow from X to Y if X is a direct cause of Y. Causes should be parents of effects.
7. **Specify CPTs:** For each variable, specify $P(X_i \mid \text{Parents}(X_i))$ for all parent combinations.
8. **Verify:** Check that the network is a DAG (no directed cycles) and CPT rows sum to 1.

□ Design Principle

In a well-constructed Bayesian network, each node should be conditionally independent of its non-descendants given its parents. A common mistake is introducing spurious dependencies by adding unnecessary arrows.

7.4 Inference in Bayesian Networks

Inference means answering probabilistic queries given evidence.

Types of Queries

Query Type	Example	Description
Diagnostic (bottom-up)	$P(\text{Burglary} \mid \text{JohnCalls}=\text{T})$	From effects to causes
Causal (top-down)	$P(\text{JohnCalls} \mid \text{Burglary}=\text{T})$	From causes to effects
Intercausal	$P(\text{Burglary} \mid \text{Alarm}=\text{T}, \text{Earthquake}=\text{T})$	Between causes of a common effect
Mixed	$P(\text{Alarm} \mid \text{Burglary}=\text{T}, \text{MaryCalls}=\text{F})$	Mix of evidence and query types

Exact Inference – Enumeration Algorithm

```
function ENUMERATION-ASK(X, e, bn):
    // X = query variable, e = evidence
    Q ← probability distribution over X
    for each value  $x_i$  of X:
         $Q(x_i) \leftarrow \text{ENUMERATE-ALL}(\text{bn.VARS}, e_{x_i})$ 
    return NORMALISE(Q)

function ENUMERATE-ALL(vars, e):
    if vars is empty: return 1.0
    Y ← FIRST(vars)
    rest ← REST(vars)
    if Y has value y in e:
        return  $P(y \mid \text{parents}(Y)) \times \text{ENUMERATE-ALL}(\text{rest}, e)$ 
    else:
        return  $\sum_y P(y \mid \text{parents}(Y)) \times \text{ENUMERATE-ALL}(\text{rest}, e_y)$ 
```

Variable Elimination

More efficient than enumeration. Key idea: **push sums inward** past products to avoid redundant computation. Each factor is computed once and reused.

```
 $P(B \mid j, m) = \alpha \times \sum_e \sum_a P(B) \times P(e) \times P(a \mid B, e) \times P(j \mid a) \times P(m \mid a)$ 

Variable elimination order (inner to outer summation):
1. Eliminate E first:  $f_E(B, a) = \sum_e P(e) \times P(a \mid B, e)$ 
2. Eliminate A:  $f_A(B) = \sum_a f_E(B, a) \times P(j \mid a) \times P(m \mid a)$ 
3. Result:  $\alpha \times P(B) \times f_A(B)$ 
```

7.5 Markov Blankets

The **Markov Blanket** of a node X is the minimal set of nodes that **shields X from the rest of the network**. Once the Markov Blanket is known, X is conditionally independent of all other nodes.

□ Markov Blanket Definition

The Markov Blanket of node X consists of: 1. Parents of X (direct causes) 2. Children of X (direct effects) 3. Other parents of X 's children (co-parents / spouses) Formal: $X \perp (\text{all other nodes}) \mid \text{MarkovBlanket}(X)$

Markov Blanket in the Alarm Network

Markov Blanket of Alarm: Parents: {Burglary, Earthquake}, Children: {JohnCalls, MaryCalls}, Co-parents: none. So $\text{MB}(\text{Alarm}) = \{\text{Burglary, Earthquake, JohnCalls, MaryCalls}\}$

Significance: To determine the probability of Alarm, we only need to observe its Markov Blanket — not the entire network.

Why Markov Blankets Matter

- Enables local computation: only a small neighbourhood of nodes is needed.
- Foundation of Gibbs Sampling (approximate inference): sample each variable given its Markov Blanket.
- Useful for learning network structure from data.
- Reduces inference complexity in large networks.

CHAPTER 8: Exam-Type Questions with Solutions

Section A: Intelligent Agents

Q1. Define a rational agent. Is a rational agent the same as an omniscient agent? Explain with an example.

Answer

A rational agent is one that selects actions to maximise its expected performance measure, given its percept sequence and any built-in prior knowledge. Rationality $\neq$ Omniscience: • Omniscient = knows the actual outcome of all actions (impossible in practice). • Rational = makes the best decision possible given available information. Example: A chess-playing agent that always selects the move with the highest expected score (based on heuristic evaluation) is rational, even if it doesn't know for certain which move will lead to a win. A human who looks both ways before crossing the road is rational; if they don't look but happen to survive, they were lucky, not rational.

Q2. Describe the PEAS framework for an autonomous vacuum-cleaning robot.

Answer

P (Performance): Amount of dirt cleaned per unit time, energy consumed, noise level. E (Environment): Room with hard floor & carpet, furniture as obstacles, different lighting conditions, dirt scattered randomly. A (Actuators): Drive wheels (left/right), suction motor (on/off), brush motor. S (Sensors): Dirt sensor (detects dirt under robot), bump sensor (detects collisions), optical sensor (detects cliffs), location sensor (e.g., IR or GPS).

Q3. Classify the following environments along the key dimensions:

a) Chess b) Poker c) Self-driving car d) Medical diagnosis system

Answer

a) Chess: Fully observable, multi-agent (competitive), deterministic, sequential, static (during deliberation), discrete, known. b) Poker: Partially observable, multi-agent, stochastic (card distribution), sequential, static (during turn), discrete, known. c) Self-driving car: Partially observable, multi-agent, stochastic, sequential, dynamic, continuous, partially known. d) Medical diagnosis: Partially observable, single-agent, stochastic, sequential, dynamic (patient condition changes), continuous, unknown (incomplete medical knowledge).

Section B: Backtracking Search

Q4. Write the BACKTRACK algorithm and explain what each component does.

Answer – Key Components

• SELECT-UNASSIGNED-VARIABLE: Chooses which variable to assign next. Default: static order. Better: MRV heuristic. • ORDER-DOMAIN-VALUES: Orders the values to try for the chosen variable. Better: LCV heuristic. • INFERENCE: Performs domain reduction after each assignment. Options: forward checking, MAC (AC-3). The algorithm is essentially DFS on a partial assignment tree. It assigns one variable per level, checks consistency at each assignment, and backtracks when a dead end is reached.

Q5. Trace the backtracking search for 4-Queens, showing where backtracking occurs.

Answer

Variables Q_1, Q_2, Q_3, Q_4 (queen per column). Domain $\{1,2,3,4\}$. Search trace: $Q_1 = 1$: OK $Q_2 = 1$: FAIL (same row as Q_1) $Q_2 = 2$: FAIL (diagonal with Q_1 : $|1-2|=|1-2|=1$) $Q_2 = 3$: OK (continue) $Q_3 = 1$: FAIL (same row as Q_1) $Q_3 = 2$: FAIL (diagonal with Q_2 : $|3-2|=|2-3|=1$) $Q_3 = 3$: FAIL

(same row as Q_2) $Q_3 = 4$: OK (continue) $Q_4 = 1$: FAIL (same row as Q_1) $Q_4 = 2$: OK ✓
 SOLUTION: $\{Q_1 = 1, Q_2 = 3, Q_3 = 4, Q_4 = 2\}$

Q6. Compare MRV, Degree Heuristic, and LCV. When would you use each?

Heuristic	When to Use	Effect
MRV	Choosing which variable to assign next (general)	Detects failures early; reduces search tree size
Degree Heuristic	Tie-breaking when MRV gives equal-size domains	Reduces future branching; useful at start of search
LCV	Choosing which value to assign to selected variable	Increases probability of finding solution without backtracking

Section C: Forward Checking & Arc Consistency

Q7. Explain the difference between Forward Checking and MAC. Which is stronger and why?

□ Answer

Forward Checking (FC): When variable X is assigned value v , remove from each neighbour Y 's domain any value inconsistent with v . Only checks X 's direct neighbours. MAC (Maintaining Arc Consistency): After assigning $X=v$, run AC-3 from all arcs involving X 's neighbours. This propagates constraints transitively through the network. MAC is strictly stronger because: 1. FC detects conflicts only 1 level deep (assigned $\leftrightarrow$ unassigned). 2. MAC propagates conflicts through the entire network. 3. MAC can detect failure earlier, preventing deeper (wasted) search. Example: FC might not detect that a chain of 3 variables is inconsistent; MAC will.

Q8. Trace AC-3 on the constraint graph where: $X_1 \in \{1,2\}$, $X_2 \in \{1,2\}$, $X_3 \in \{1,2\}$, with constraints $X_1 \neq X_2$, $X_2 \neq X_3$.

□ Answer

Initial arcs in queue: $\{(X_1, X_2), (X_2, X_1), (X_2, X_3), (X_3, X_2)\}$ Process (X_1, X_2) : For $X_1 = 1$: $X_2 = 2$ is consistent ✓; $X_1 = 2$: $X_2 = 1$ is consistent ✓. No revision. Process (X_2, X_1) : Similarly, no revision. Process (X_2, X_3) : For $X_2 = 1$: $X_3 = 2$ consistent ✓; $X_2 = 2$: $X_3 = 1$ consistent ✓. No revision. Process (X_3, X_2) : No revision. Result: No domains reduced. Network is already arc-consistent. Solutions exist: $\{1,2,1\}$ and $\{2,1,2\}$. Key insight: AC-3 returns true (consistent) but doesn't solve the problem alone — search is still needed.

Section D: Probability

Q9. Using the full joint distribution (Toothache, Cavity, Catch), compute $P(\text{cavity} \mid \text{toothache})$.

□ Answer

Joint entries involving toothache: $P(\text{toothache}, \text{cavity}, \text{catch}) = 0.108$ $P(\text{toothache}, \text{cavity}, \neg \text{catch}) = 0.012$ $P(\text{toothache}, \neg \text{cavity}, \text{catch}) = 0.016$ $P(\text{toothache}, \neg \text{cavity}, \neg \text{catch}) = 0.064$ $P(\text{toothache}) = 0.108 + 0.012 + 0.016 + 0.064 = 0.200$ $P(\text{toothache} \wedge \text{cavity}) = 0.108 + 0.012 = 0.120$ $P(\text{cavity} \mid \text{toothache}) = P(\text{toothache} \wedge \text{cavity}) / P(\text{toothache}) = 0.120 / 0.200 = 0.60$ So given a toothache, there is a 60% chance of a cavity.

Q10. Apply Bayes' Rule: A disease affects 0.1% of the population. A test is 99% sensitive and 95% specific. What is $P(\text{disease} \mid \text{test positive})$?

□ Answer

Given: • $P(D) = 0.001$ (prior: 0.1% have disease) • $P(+|D) = 0.99$ (sensitivity) • $P(+|\neg D) = 0.05$ (false positive rate; specificity = 95%) Using Bayes' Rule: $P(D|+) = P(+|D) \times P(D) / P(+)$ $P(+)$ = $P(+|D) \times P(D) + P(+|\neg D) \times P(\neg D)$ = $0.99 \times 0.001 + 0.05 \times 0.999$ = $0.00099 + 0.04995$ = 0.05094 $P(D|+) = (0.99 \times 0.001) / 0.05094$ = $0.00099 / 0.05094$ $\approx 0.0194 \approx 1.94\%$ Conclusion: Only ~2% chance of disease despite positive test. The low prior overwhelms even a good test!

Section E: Bayesian Networks

Q11. What is a Bayesian network? List its components and explain the role of CPTs.

□ Answer

A Bayesian network is a directed acyclic graph (DAG) representing probabilistic relationships among variables. Components: 1. Nodes: Each node represents a random variable. 2. Edges: A directed edge $X \rightarrow Y$ means X is a direct cause/influence of Y . 3. CPTs (Conditional Probability Tables): Each node X_i has a CPT $P(X_i | \text{Parents}(X_i))$ specifying the probability distribution of X_i for each combination of parent values. Role of CPTs: Together with the graph structure, CPTs implicitly define the full joint distribution via: $P(x_1, \dots, x_n) = \prod P(x_i | \text{parents}(X_i))$. Key property: Each variable is conditionally independent of all non-descendants given its parents (Markov condition).

Q12. Define the Markov Blanket of a node. Identify the Markov Blanket of JohnCalls in the Alarm network.

□ Answer

The Markov Blanket of node X consists of: (1) X 's parents, (2) X 's children, (3) other parents of X 's children. Alarm network structure: Burglary $\rightarrow$ Alarm $\leftarrow$ Earthquake Alarm $\rightarrow$ JohnCalls Alarm $\rightarrow$ MaryCalls Markov Blanket of JohnCalls: • Parents of JohnCalls: {Alarm} • Children of JohnCalls: none • Co-parents of JohnCalls's children: none MB(JohnCalls) = {Alarm} Significance: JohnCalls is conditionally independent of {Burglary, Earthquake, MaryCalls} given Alarm.

Q13. Using the Alarm Bayesian Network, compute $P(\text{Burglary} | \text{JohnCalls}=\text{T}, \text{MaryCalls}=\text{T})$.

□ Answer – Using Enumeration

$P(B|j,m) = \alpha \times P(B) \times \sum_e \sum_a P(e) \times P(a|B,e) \times P(j|a) \times P(m|a)$ For $P(b=\text{true}|j,m)$: = $\alpha \times 0.001 \times [P(a|b,e) \times P(e) \times P(j|a) \times P(m|a)]$ summed over e,a Computing key terms: $e=\text{T}, a=\text{T}$: $0.002 \times 0.95 \times 0.90 \times 0.70 = 0.001197$ $e=\text{T}, a=\text{F}$: $0.002 \times 0.05 \times 0.05 \times 0.01 = 0.0000005$ $e=\text{F}, a=\text{T}$: $0.998 \times 0.94 \times 0.90 \times 0.70 = 0.592719\dots$ $e=\text{F}, a=\text{F}$: $0.998 \times 0.06 \times 0.05 \times 0.01 = 0.0000030\dots$ Summing: $P(b,j,m) \approx 0.001 \times 0.5939 \approx 0.000594$ Similarly for $P(\neg b,j,m) \approx 0.001494$ $\alpha = 1/(0.000594 + 0.001494) = 1/0.002088$ $P(B=\text{true} | j, m) \approx 0.000594/0.002088 \approx 0.284 \approx 28.4\%$

End of Notes — Good luck with your exams! □